BTS CIEL · PROGRAMMATION C++ · TRAVAUX PRATIQUES

TP05 — Décodeur de trames de capteurs

Prérequis : chapitres 1 à 6

Contexte professionnel

Des capteurs sans fil transmettent leurs mesures à une passerelle sous forme de **trames binaires**. Pour limiter la lecture des données par un tiers sur le lien radio, la partie « données » de chaque trame est **masquée par XOR** avec une clé de 4 octets connue de la passerelle. Une **somme de contrôle** permet de détecter les trames altérées pendant la transmission.

Vous développez le module de décodage de la passerelle : il vérifie chaque trame reçue, démasque les données et affiche la mesure, ou un diagnostic précis en cas d'erreur.

Objectifs

- Manipuler des octets (`std::uint8_t`) et les afficher en hexadécimal.
- Vérifier l'intégrité d'une trame : en-tête, longueur, somme de contrôle par XOR.
- Démasquer des données avec une clé répétée.
- Convertir des caractères ASCII en valeur numérique.
- Organiser le programme en fonctions testables.

Compétences mobilisées

Compétence	Mise en œuvre dans ce TP
C04 — Analyser un système informatique	Décoder une trame à la main à partir de sa spécification
C05 — Concevoir un système informatique	Découper le décodage en fonctions et définir les codes d'erreur
C06 — Valider un système informatique	Tester le décodeur sur des trames valides et volontairement erronées
C08 — Coder	Écrire les fonctions de vérification, de démasquage et de conversion

Spécification de la trame

Rang de l'octet	Champ	Contenu
1	En-tête	Toujours `0xAA`
2	Adresse	Adresse du capteur (`0x00` à `0xFF`)
3	Type	`0x01` : température (°C) ; `0x02` : humidité (%)
4	Longueur N	Nombre d'octets de données (1 à 16)
5 à 4 + N	Données	Texte ASCII **masqué** : la mesure en **dixièmes**, avec un `-` éventuel (`215` = 21,5 ; `-85` = -8,5)
5 + N (dernier)	Somme de contrôle	XOR de **tous** les octets précédents, en-tête compris

- Clé de masquage : `5A 3C A5 0F`, répétée. Le premier octet de **données** utilise le premier octet de clé.
- Une trame complète compte donc exactement N + 5 octets.

Codes de vérification

Code	Signification	Message affiché
0	Trame valide	—
1	Moins de 5 octets	`Erreur : trame trop courte`
2	Premier octet différent de `0xAA`	`Erreur : en-tete invalide`
3	Taille différente de N + 5	`Erreur : longueur incoherente`
4	Somme recalculée différente de la somme reçue	`Erreur : somme de controle incorrecte (calculee 0x.., recue 0x..)`

Les contrôles sont effectués **dans cet ordre** ; on s'arrête au premier échec.

Travail demandé

Partie 1 — Analyse

1. Décodez à la main la trame `AA 12 01 03 68 0D 90 4F` : identifiez chaque champ, recalculez la somme de contrôle (XOR posés en binaire), démasquez les trois octets de données et donnez la mesure.
2. Pourquoi la vérification de longueur doit-elle précéder le calcul de la somme ?

Partie 2 — Fonctions de vérification

3. Complétez le fichier de départ `decodeur.cpp`, qui contient les six trames de test. Écrivez :
   - `afficherTrame` : affiche les octets en hexadécimal sur deux chiffres, séparés par des espaces ;
   - `calculerSomme` : XOR de tous les octets sauf le dernier ;
   - `verifierTrame` : renvoie le code de vérification.

Partie 3 — Données

4. Écrivez `demasquerDonnees`, qui renvoie le texte démasqué d'une trame valide.
5. Écrivez `texteVersDixiemes(const std::string& texte, int& dixiemes)` : elle convertit un texte comme `"215"` ou `"-85"` en entier, sans fonction de la bibliothèque, à partir des codes ASCII. Elle renvoie `false` si le texte est vide, réduit à `-`, ou contient un caractère qui n'est pas un chiffre.

Partie 4 — Programme complet

6. Pour chaque trame : affichez-la, vérifiez-la, puis affichez la mesure ou le message d'erreur. Terminez par un bilan.
7. Ajoutez dans `main()` les tests unitaires de `texteVersDixiemes` du tableau ci-dessous.

Contraintes techniques

- Compilation avec `g++ -std=c++17 -Wall -Wextra`, sans erreur ni avertissement.
- Octets de type `std::uint8_t`, convertis en `unsigned int` pour l'affichage.
- En-tête, clé, codes d'erreur, types de mesure, tailles minimale et maximale en constantes.
- Les trames sont passées aux fonctions par **référence constante**.
- Aucun accès hors limites : vérifiez la taille d'une trame **avant** de lire ses octets.

Tests à effectuer

Trames (fichier de départ)

N°	Trame	Résultat attendu
1	`AA 12 01 03 68 0D 90 4F`	`Capteur 0x12 : temperature = 21.5 C`
2	`AA 07 02 03 6F 0A 96 5F`	`Capteur 0x07 : humidite = 56.3 %`
3	`AA 21 01 03 77 04 90 6A`	`Capteur 0x21 : temperature = -8.5 C`
4	`AA 12 01 03 68 0D 90 4E`	`Erreur : somme de controle incorrecte (calculee 0x4F, recue 0x4E)`
5	`AB 12 01 03 68 0D 90 4E`	`Erreur : en-tete invalide`
6	`AA 07 02 04 6F 0A 96 58`	`Erreur : longueur incoherente`

Bilan attendu : `Bilan : 3 trame(s) valide(s), 3 en erreur`.

Tests unitaires de `texteVersDixiemes`

Texte	Retour	`dixiemes`
`"215"`	`true`	`215`
`"-85"`	`true`	`-85`
`"0"`	`true`	`0`
`""`	`false`	—
`"-"`	`false`	—
`"2a5"`	`false`	—

Pour aller plus loin : ajoutez une trame trop courte (`AA 12`) et une trame de type inconnu (`0x03`) ; prévoyez puis vérifiez le comportement du décodeur.

Livrables

- Le fichier `decodeur.cpp` complété.
- La feuille d'analyse (partie 1) et le compte rendu des tests.

Critères d'évaluation

Critère	Indicateurs de réussite	Points
Analyse	Trame 1 décodée à la main sans erreur ; ordre des contrôles justifié	4
Vérification	Codes 1 à 4 détectés dans l'ordre, aucun accès hors limites	5
Démasquage et conversion	Clé répétée correctement indexée ; conversion ASCII sans bibliothèque, cas invalides traités	5
Affichage	Octets sur deux chiffres hexadécimaux, mesures et diagnostics conformes, bilan	3
Qualité et tests	Fonctions, références constantes, constantes, tests unitaires présents	3
Total		**20**